

Sprint 12: WAI-ARIA Essentials and Final HTML Capstone

Sprint 12 Goal

The goal of Sprint 12 is to understand **WAI-ARIA essentials** and use them carefully in a final HTML capstone project.

By the end of this sprint, you should understand:

- What WAI-ARIA is
- Why ARIA exists
- The difference between WCAG and WAI-ARIA
- Why native HTML should be used before ARIA
- What ARIA roles, states, and properties are
- Why ARIA roles do not add real behavior
- When to use `aria-label`
- When to use `aria-labelledby`
- When to use `aria-describedby`
- When to use `aria-hidden`
- Why `tabindex` and `accesskey` must be used carefully
- How to avoid common ARIA mistakes
- How to build a final accessible HTML capstone page

1. Where You Are Now

You have already completed Sprints 1 to 11.

That means you already learned the main HTML foundation.

Sprint	Main Topic You Learned
Sprint 1	HTML elements, tags, nesting, attributes, void elements, <code>id</code> , <code>class</code>
Sprint 2	Full HTML boilerplate, <code>head</code> , metadata, <code>body</code>
Sprint 3	Text meaning with elements like <code>em</code> , <code>strong</code> , <code>abbr</code> , <code>time</code> , <code>code</code> , <code>pre</code>
Sprint 4	Links, paths, <code>mailto:</code> , <code>tel:</code> , navigation
Sprint 5	Images, audio, video, SVG, <code>iframe</code> , media responsibility
Sprint 6	Semantic layout: <code>header</code> , <code>nav</code> , <code>main</code> , <code>section</code> , <code>article</code> , <code>footer</code>

Sprint	Main Topic You Learned
Sprint 7	Form basics, labels, inputs, placeholders
Sprint 8	Buttons, validation, disabled, readonly, focus states
Sprint 9	Tables, captions, table structure, debugging, validators
Sprint 10	Accessibility foundations, WCAG, POUR, assistive technology
Sprint 11	Accessible HTML patterns: headings, labels, tables, alt text, links, media, keyboard navigation
Sprint 12	WAI-ARIA essentials and final HTML capstone

Now Sprint 12 asks a final question:

Can I build a valid, semantic, accessible HTML page and use ARIA only when needed?

Simple idea:

Sprints 1 to 11 taught you how to write good HTML. Sprint 12 teaches you how to use ARIA carefully without damaging accessibility.

2. What Is WAI-ARIA?

WAI-ARIA means:

Web Accessibility Initiative — Accessible Rich Internet Applications

In simple words:

WAI-ARIA is a set of special accessibility attributes that can help assistive technologies understand parts of a web page.

Assistive technologies include tools such as:

- Screen readers
- Braille output devices
- Voice control software
- Browser accessibility tools
- Other accessibility software

ARIA can help when normal HTML does not give enough information.

Example:

```
<button type="button" aria-label="Search">  
  <span aria-hidden="true">🔍</span>  
</button>
```

The user sees only a search icon.

A screen reader may not understand the icon clearly.

So `aria-label="Search"` gives the button an accessible name.

Simple rule:

ARIA gives extra accessibility information when native HTML is not enough.

3. WAI-ARIA vs WCAG

WAI-ARIA and WCAG are related, but they are not the same.

WCAG

WCAG means:

Web Content Accessibility Guidelines

WCAG gives accessibility goals and rules.

WCAG tells us what accessible content should achieve.

Example WCAG idea:

Users should be able to understand and use the page.

WCAG is built around the POUR principles:

Principle	Meaning
Perceivable	Users must be able to receive the information
Operable	Users must be able to use the interface
Understandable	Users must understand the content and controls

Principle	Meaning
Robust	Browsers and assistive technologies must understand the code

WAI-ARIA

WAI-ARIA gives specific attributes that can help assistive technologies.

Examples:

```
role
aria-label
aria-labelledby
aria-describedby
aria-hidden
aria-expanded
aria-modal
```

Simple comparison:

Item	Simple Meaning
WCAG	Accessibility rules and goals
WAI-ARIA	Special attributes used when needed

Simple memory rule:

WCAG tells you what accessibility should achieve. ARIA gives extra tools to help when HTML alone is not enough.

4. The Most Important ARIA Rule

The most important ARIA rule is:

Use native HTML first. Use ARIA only when needed.

This is sometimes explained as:

No ARIA is better than bad ARIA.

ARIA can help accessibility, but wrong ARIA can also break accessibility.

5. Native HTML First

Native HTML means using the correct HTML element that already has built-in meaning and behavior.

Examples of native HTML elements:

```
<button>
<a>
<nav>
<main>
<header>
<footer>
<form>
<label>
<input>
<table>
```

These elements already have meaning.

Many of them also have built-in keyboard behavior.

For example, a real button can be focused with the keyboard and activated using keyboard keys.

6. Bad Example: Fake Button with ARIA

Bad:

```
<div role="button">Search</div>
```

This is weak because:

- `div` is not naturally a button
- `role="button"` only tells assistive technology that it is meant to be a button
- It does not automatically add real button behavior
- It may not work correctly with keyboard users

Better:

```
<button type="button">Search</button>
```

Why better?

- `button` already has button meaning
- `button` already supports keyboard interaction
- `button` is understood by browsers and assistive technology
- The code is simpler and safer

Simple rule:

Do not use ARIA to turn the wrong element into the right element when the right HTML element already exists.

7. ARIA Does Not Add Behavior

This is a very important beginner rule:

ARIA can describe meaning, but it does not create behavior.

Example:

```
<div role="button">Submit</div>
```

`role="button"` tells assistive technology:

This should be treated like a button.

But it does not automatically add:

- Click behavior
- Keyboard Enter behavior
- Keyboard Space behavior
- Form submission behavior
- Disabled behavior
- Focus management

A real button already has these behaviors.

Correct:

```
<button type="submit">Submit</button>
```

Simple memory rule:

ARIA gives meaning. HTML gives meaning and behavior.

8. Roles, States, and Properties

ARIA has three main ideas:

1. Roles
 2. States
 3. Properties
-

8.1 ARIA Roles

A role tells assistive technology what something is.

Example:

```
<div role="dialog">
  <h2>Delete account?</h2>
  <p>This action cannot be undone.</p>
</div>
```

Here, `role="dialog"` tells assistive technology:

This part is a dialog.

But remember:

The role does not create the full dialog behavior by itself.

It does not automatically handle:

- Keyboard focus
- Escape key closing
- Background blocking
- Focus trapping
- Opening and closing behavior

So, ARIA roles must be used carefully.

8.2 ARIA States

A state describes the current condition of something.

Example:

```
<button type="button" aria-expanded="false">
  Menu
</button>
```

This tells assistive technology:

The menu is currently collapsed.

If the menu is open, the value may become:

```
aria-expanded="true"
```

Simple meaning:

State	Simple Meaning
<code>aria-expanded="false"</code>	The related content is closed or collapsed
<code>aria-expanded="true"</code>	The related content is open or expanded

Important beginner note:

In real websites, changing this value usually needs JavaScript. In this HTML-only sprint, you only need to understand the meaning.

8.3 ARIA Properties

A property gives extra information about an element.

Example:

```
<input
  type="password"
  id="password"
  name="password"
  aria-describedby="password-help"
```

```
>  
  
<p id="password-help">  
  Your password must be at least 8 characters long.  
</p>
```

Here, `aria-describedby` tells assistive technology:

This input has extra description text in the element with `id="password-help"`.

Simple rule:

ARIA properties connect extra information to an element.

9. ARIA Role Categories

ARIA roles are grouped into categories.

The main categories mentioned in Sprint 12 are:

1. Document structure roles
2. Widget roles
3. Landmark roles
4. Live region roles
5. Window roles
6. Abstract roles

You do not need to master every role in Sprint 12.

You need to understand what these categories mean at a beginner level.

9.1 Document Structure Roles

Document structure roles describe parts of document content.

They can describe things like:

- Articles
- Headings
- Lists
- Tables
- Notes

But in normal beginner HTML, native HTML often already gives this structure.

Example:

Use this:

```
<article>
  <h2>HTML Basics</h2>
  <p>HTML gives structure to web pages.</p>
</article>
```

Do not unnecessarily write:

```
<div role="article">
  <h2>HTML Basics</h2>
  <p>HTML gives structure to web pages.</p>
</div>
```

Simple rule:

Use semantic HTML elements before ARIA document roles.

9.2 Widget Roles

Widget roles describe interactive controls.

Examples include things like:

- Button
- Checkbox
- Slider
- Tab
- Menu
- Dialog-related controls

But many widgets already have native HTML elements.

Example:

Better:

```
<button type="button">Open Menu</button>
```

Weaker:

```
<div role="button">Open Menu</div>
```

Simple rule:

Use native interactive elements before ARIA widget roles.

9.3 Landmark Roles

Landmark roles describe major areas of a page.

Examples of landmark ideas:

- Navigation
- Main content
- Banner/header area
- Footer/contentinfo area
- Complementary/aside content

But HTML already has native landmark elements.

Use these first:

```
<header>  
<nav>  
<main>  
<aside>  
<footer>  
<form>
```

Good:

```
<nav>  
  <a href="index.html">Home</a>  
  <a href="courses.html">Courses</a>  
</nav>
```

Usually unnecessary:

```
<div role="navigation">
  <a href="index.html">Home</a>
  <a href="courses.html">Courses</a>
</div>
```

Simple rule:

Native landmarks are cleaner than ARIA landmarks.

9.4 Live Region Roles

Live region roles are used when page content changes dynamically and users need to be informed.

Example idea:

- New notification appears
- Error message appears
- Success message appears
- Cart count changes

A common ARIA live region pattern looks like:

```
<p aria-live="polite">Your profile was saved.</p>
```

Simple meaning:

Assistive technology should announce this update politely when it changes.

Important beginner note:

Live regions usually matter more when JavaScript changes content. This curriculum does not teach DOM manipulation, so you only need to recognize the idea.

9.5 Window Roles

Window roles describe interface windows or dialog-like content.

Example:

```
<div role="dialog" aria-labelledby="dialog-title">
  <h2 id="dialog-title">Confirm Delete</h2>
```

```
<p>Are you sure you want to delete this item?</p>
</div>
```

This says:

This content is a dialog, and its label is the heading with `id="dialog-title"`.

But remember:

`role="dialog"` alone does not create a fully accessible dialog.

A real accessible dialog also needs correct behavior such as focus management.

Simple rule:

Dialog ARIA is powerful, but it must be used carefully.

9.6 Abstract Roles

Abstract roles are not for normal HTML developers to use directly.

They are mostly used internally in ARIA specifications.

Simple rule:

Do not use abstract roles in your own HTML.

10. Native Landmarks Before ARIA Landmarks

Before adding ARIA roles, ask:

Does HTML already have an element for this?

If yes, use the HTML element.

Good Native Page Structure

```
<header>
  <h1>HTML Course Catalog</h1>
  <nav>
    <a href="index.html">Home</a>
```

```

    <a href="courses.html">Courses</a>
    <a href="contact.html">Contact</a>
  </nav>
</header>

<main>
  <section>
    <h2>Featured Courses</h2>
    <p>Choose a beginner-friendly HTML course.</p>
  </section>
</main>

<footer>
  <p>&copy; 2026 HTML Learning Hub</p>
</footer>

```

This is good because:

- `header` gives introductory page area
- `nav` gives navigation area
- `main` gives main content area
- `section` gives a meaningful page section
- `footer` gives footer content

No ARIA is needed here.

11. `aria-label`

`aria-label` gives an accessible name directly in the attribute.

Use it when an element needs a label but does not have visible text.

Good Use Case: Icon-Only Button

```

<button type="button" aria-label="Search">
  <span aria-hidden="true">🔍</span>
</button>

```

The user sees:



The screen reader gets:

Search

This is useful because the icon alone may not be clear to assistive technology.

When to Use `aria-label`

Use `aria-label` when:

- The control has no visible text
- The visible content is only an icon
- The icon needs a clear accessible name
- There is no visible label to reference with `aria-labelledby`

Example:

```
<a href="settings.html" aria-label="Settings">  
  <span aria-hidden="true">⚙️</span>  
</a>
```

This link only shows a gear icon.

`aria-label="Settings"` tells screen readers what the link does.

When Not to Use `aria-label`

Do not use `aria-label` when visible text already explains the element.

Unnecessary:

```
<button type="submit" aria-label="Submit form">Submit form</button>
```

Better:

```
<button type="submit">Submit form</button>
```

Why?

The visible text already gives the button name.

Simple rule:

If visible text already explains the control, you usually do not need `aria-label`.

12. `aria-labelledby`

`aria-labelledby` points to visible text somewhere else on the page.

It uses the `id` of another element.

Basic Pattern

```
<h2 id="course-title">HTML Beginner Course</h2>

<section aria-labelledby="course-title">
  <p>This course teaches HTML from the beginning.</p>
</section>
```

Meaning:

This section is labelled by the heading with `id="course-title"`.

The value must match the target `id`.

```
aria-labelledby="course-title"
```

connects to:

```
id="course-title"
```

Why `aria-labelledby` Is Useful

It is useful when:

- A visible heading already labels an area
- You want assistive technology to use that visible text as the label
- You want the accessible name to come from real visible page content

Example:

```
<h2 id="filter-heading">Course Filters</h2>

<form action="/courses" aria-labelledby="filter-heading">
  <label for="level">Course Level:</label>
  <input type="text" id="level" name="level">
</form>
```

Here, the form is labelled by the heading “Course Filters.”

aria-labelledby with Multiple IDs

aria-labelledby can reference more than one id.

Example:

```
<span id="volume-label">Volume</span>
<span id="volume-details">Adjust the volume level</span>

<input
  type="range"
  min="0"
  max="100"
  value="30"
  aria-labelledby="volume-label volume-details"
>
```

This combines the text from both elements.

The accessible name becomes something like:

Volume Adjust the volume level

Simple rule:

aria-labelledby points to visible text using one or more IDs.

13. aria-label vs aria-labelledby

Beginners often confuse these two.

Attribute	Simple Meaning	Text Comes From
<code>aria-label</code>	Gives a hidden accessible name directly	The attribute value
<code>aria-labelledby</code>	Uses visible text as the accessible name	Another element with an <code>id</code>

`aria-label` Example

```
<button type="button" aria-label="Search">
  <span aria-hidden="true">🔍</span>
</button>
```

The label text is inside the attribute.

`aria-labelledby` Example

```
<h2 id="search-heading">Search Courses</h2>

<form action="/search" aria-labelledby="search-heading">
  <label for="query">Keyword:</label>
  <input type="text" id="query" name="query">
</form>
```

The label text comes from the visible heading.

14. Do Not Use `aria-label` and `aria-labelledby` Together

Do not put both on the same element.

Weak:

```
<button
  type="button"
  aria-label="Search"
  aria-labelledby="search-text"
>
  <span id="search-text">Find</span>
</button>
```

Why weak?

- The element has two possible accessible names
- `aria-labelledby` usually wins
- `aria-label` may be ignored
- The code becomes confusing

Better:

```
<button type="button">
  <span id="search-text">Find</span>
</button>
```

No ARIA is needed because visible text already exists.

Simple rule:

Use either `aria-label` or `aria-labelledby`, not both on the same element.

15. `aria-describedby`

`aria-describedby` connects an element to extra description text.

This is useful for:

- Help text
- Hints
- Rules
- Error messages
- Warnings
- Extra instructions

Basic Pattern

```
<label for="password">Password:</label>
<input
  type="password"
  id="password"
  name="password"
  aria-describedby="password-help"
>

<p id="password-help">
```

```
    Your password must be at least 8 characters long.
  </p>
```

Meaning:

The password input is described by the paragraph with `id="password-help"`.

Why `aria-describedby` Is Useful

A label tells the user what the field is.

A description gives extra help.

Example:

```
<label for="email">Email Address:</label>
<input
  type="email"
  id="email"
  name="email"
  aria-describedby="email-help"
>

<p id="email-help">
  We will use this email to send your course confirmation.
</p>
```

Here:

- Label = Email Address
- Description = We will use this email to send your course confirmation

Simple rule:

`label` gives the field name. `aria-describedby` connects extra help text.

Common `aria-describedby` Mistake

Bad:

```
<input type="password" aria-describedby="help-text">
<p id="password-help">Use 8 characters.</p>
```

Problem:

```
aria-describedby="help-text"
```

But the paragraph has:

```
id="password-help"
```

The names do not match.

So the connection is broken.

Better:

```
<label for="password">Password:</label>
<input
  type="password"
  id="password"
  name="password"
  aria-describedby="password-help"
>

<p id="password-help">Use at least 8 characters.</p>
```

Simple rule:

The `aria-describedby` value must match the `id` of the description element.

16. `aria-hidden`

`aria-hidden="true"` hides an element from assistive technology.

Important:

It does not hide the element visually.

The element can still appear on the screen.

It is only removed from the accessibility tree.

Good Use Case: Decorative Icon

```
<button type="button">
  <span aria-hidden="true">⚙️</span>
  <span>Settings</span>
</button>
```

Here, the gear icon is decorative.

The visible text “Settings” already explains the button.

So the screen reader does not need to announce the icon.

Icon-Only Button with `aria-hidden`

```
<button type="button" aria-label="Search">
  <span aria-hidden="true">🔍</span>
</button>
```

Here:

- `aria-label="Search"` gives the button a name
- `aria-hidden="true"` hides the decorative icon from assistive technology

This prevents the screen reader from trying to announce the icon itself.

Do Not Use `aria-hidden` to Hide Content from Everyone

Bad idea:

```
<p aria-hidden="true">Important deadline: Submit by Friday.</p>
```

Why bad?

The text is still visible on screen, but screen reader users may not receive it.

If the information is important, do not hide it from assistive technology.

Simple rule:

Do not use `aria-hidden` on meaningful content.

Do Not Use `aria-hidden` on Focusable Elements

Bad:

```
<a href="settings.html">
  <span aria-hidden="true" tabindex="0">⌘</span>
</a>
```

Why bad?

The icon is hidden from assistive technology, but it can still receive keyboard focus.

This creates confusion because keyboard users may focus something that assistive technology does not announce correctly.

Better:

```
<a href="settings.html" aria-label="Settings">
  <span aria-hidden="true">⌘</span>
</a>
```

Simple rule:

Never hide focusable content from assistive technology.

17. `tabindex` Caution

`tabindex` controls keyboard focus order.

You learned this in Sprint 11, but it is important again in Sprint 12 because ARIA and keyboard accessibility are connected.

Natural Tab Order

Normally, users move through focusable elements in the order they appear in the HTML.

Example:

```
<a href="index.html">Home</a>  
<a href="courses.html">Courses</a>  
<button type="submit">Submit</button>
```

The user tabs through:

1. Home
2. Courses
3. Submit

This is natural and predictable.

Simple rule:

Good HTML order usually creates good keyboard order.

`tabindex="0"`

`tabindex="0"` adds an element to the normal tab order.

Example:

```
<div tabindex="0">Focusable custom area</div>
```

This means the element can receive keyboard focus.

But be careful.

Do not add focus to things that are not interactive.

Simple rule:

Use `tabindex="0"` only when there is a strong accessibility reason.

`tabindex="-1"`

`tabindex="-1"` removes an element from normal Tab navigation, but it can still be focused programmatically.

Example:

```
<h2 id="main-heading" tabindex="-1">Main Content</h2>
```

This is sometimes used for skip links or focus management.

Beginner note:

You only need to recognize this pattern. You do not need to build JavaScript focus behavior in this HTML-only curriculum.

Positive `tabindex`

Bad:

```
<input type="text" tabindex="3">  
<input type="email" tabindex="1">  
<button type="submit" tabindex="2">Submit</button>
```

Why bad?

Positive `tabindex` values create a custom tab order.

This can make keyboard navigation confusing.

The visual order and keyboard order may not match.

Better:

```
<input type="text">  
<input type="email">  
<button type="submit">Submit</button>
```

Simple rule:

Avoid positive `tabindex` values.

18. `accesskey` Caution

`accesskey` creates keyboard shortcuts for elements.

Example:

```
<a href="index.html" accesskey="h">Home</a>
```

This may seem useful, but it can cause problems.

Why?

Because shortcuts may conflict with:

- Browser shortcuts
- Operating system shortcuts
- Screen reader shortcuts
- User assistive technology shortcuts
- Different keyboard layouts

Simple rule:

Avoid `accesskey` unless there is a clear accessibility plan and testing.

19. Common ARIA Mistakes

Mistake 1: Using ARIA Instead of Native HTML

Bad:

```
<div role="button">Submit</div>
```

Better:

```
<button type="submit">Submit</button>
```

Mistake 2: Thinking ARIA Adds Behavior

Bad thinking:

I added `role="button"`, so now it works like a button.

Correct thinking:

ARIA describes meaning. It does not create real behavior.

Mistake 3: Adding ARIA When Visible Text Is Enough

Unnecessary:

```
<button type="submit" aria-label="Send Message">Send Message</button>
```

Better:

```
<button type="submit">Send Message</button>
```

Mistake 4: Using `aria-label` and `aria-labelledby` Together

Bad:

```
<button aria-label="Search" aria-labelledby="search-text">
  <span id="search-text">Find</span>
</button>
```

Better:

```
<button type="button">Find</button>
```

Mistake 5: Broken ID Reference

Bad:

```
<input aria-describedby="email-help-text">
<p id="email-help">Enter your email address.</p>
```

Problem:

The values do not match.

Better:

```
<input aria-describedby="email-help">
<p id="email-help">Enter your email address.</p>
```

Mistake 6: Hiding Important Content with `aria-hidden`

Bad:

```
<p aria-hidden="true">Your payment failed.</p>
```

Why bad?

A screen reader user may not receive this important message.

Better:

```
<p>Your payment failed.</p>
```

Mistake 7: Using `aria-hidden` on Focusable Content

Bad:

```
<button aria-hidden="true">Close</button>
```

Why bad?

The button may still be focusable, but assistive technology may ignore it.

Better:

```
<button type="button">Close</button>
```

20. Debug Sprint 12 Code

Broken Code

```
<div role="button">Search</div>

<button aria-label="Search" aria-labelledby="search-text">
  <span id="search-text">Find</span>
</button>

<a href="settings.html">
  <i aria-hidden="true" tabindex="0"></i>
</a>

<input type="password" aria-describedby="help-text">
<p id="password-help">Use 8 characters.</p>
```

Problem 1: Fake Button

Broken:

```
<div role="button">Search</div>
```

Problem:

A `div` is not a real button.

`role="button"` does not add real button behavior.

Fixed:

```
<button type="button">Search</button>
```

Problem 2: `aria-label` and `aria-labelledby` Together

Broken:

```
<button aria-label="Search" aria-labelledby="search-text">
  <span id="search-text">Find</span>
</button>
```

Problem:

The button has two accessible naming methods.

Better:

```
<button type="button">
  <span id="search-text">Find</span>
</button>
```

Even simpler:

```
<button type="button">Find</button>
```

No ARIA is needed because the button already has visible text.

Problem 3: `aria-hidden` with Focusable Content

Broken:

```
<a href="settings.html">
  <i aria-hidden="true" tabindex="0"></i>
</a>
```

Problem:

The icon is hidden from assistive technology but can receive keyboard focus because of `tabindex="0"`.

Better:

```
<a href="settings.html" aria-label="Settings">
  <span aria-hidden="true">⦿</span>
</a>
```

Here:

- The link has an accessible name: Settings
- The icon is decorative
- The icon is not focusable separately

Problem 4: Broken `aria-describedby` Reference

Broken:

```
<input type="password" aria-describedby="help-text">
<p id="password-help">Use 8 characters.</p>
```

Problem:

`aria-describedby="help-text"` does not match `id="password-help"`.

Fixed:

```
<label for="password">Password:</label>
<input
  type="password"
  id="password"
  name="password"
  aria-describedby="password-help"
>
<p id="password-help">Use at least 8 characters.</p>
```

21. Corrected Full Debug Version

```
<button type="button">Search</button>

<button type="button">Find</button>

<a href="settings.html" aria-label="Settings">
```

```
<span aria-hidden="true">⦿</span>
</a>

<label for="password">Password:</label>
<input
  type="password"
  id="password"
  name="password"
  aria-describedby="password-help"
>
<p id="password-help">Use at least 8 characters.</p>
```

Why this is better:

- Real buttons are used
- Visible text is used when available
- The icon-only link has a clear accessible name
- Decorative icon is hidden correctly
- Help text is connected correctly
- The password field has a real label

22. Final HTML Capstone

Sprint 12 ends with a final capstone project.

The capstone proves that you can combine everything from the full HTML curriculum.

Your final file should be named:

```
accessible-html-capstone.html
```

The suggested theme is:

```
Accessible Course Catalog Page
```

23. Capstone Requirements

Your capstone should include:

- Complete HTML boilerplate
- Correct `DOCTYPE`

- `html` element with `lang`
- `head` with metadata
- `meta charset="UTF-8"`
- Viewport meta tag
- Meaningful `title`
- Meta description
- Semantic page layout
- Correct heading hierarchy
- Accessible navigation
- Accessible form
- Form labels connected to inputs
- Helpful `aria-describedby` where needed
- Accessible table
- Table `caption`
- Table header cells with `scope`
- Meaningful image with useful `alt`
- Decorative image with empty `alt` if needed
- Media with captions/subtitles using `track`
- Transcript or text alternative for media
- One icon-only button or link using `aria-label`
- `aria-hidden` only for decorative content
- Minimal ARIA
- No unnecessary ARIA
- No React
- No JavaScript DOM manipulation

Simple capstone rule:

Build a full page using semantic HTML first, then add only the ARIA that is truly needed.

24. Capstone Page Structure Example

A good capstone page may follow this structure:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <meta name="description" content="Accessible HTML course catalog for beginner
web development students.">
  <title>Accessible HTML Course Catalog</title>
</head>
<body>
```

```

<header>
  <h1>Accessible HTML Course Catalog</h1>
  <nav>
    <a href="#courses">Courses</a>
    <a href="#schedule">Schedule</a>
    <a href="#register">Register</a>
  </nav>
</header>

<main>
  <section id="courses">
    <h2>Featured Courses</h2>
    <article>
      <h3>HTML Fundamentals</h3>
      <p>Learn how to structure web pages using semantic HTML.</p>
    </article>
  </section>

  <section id="schedule">
    <h2>Course Schedule</h2>
    <table>
      <caption>Upcoming HTML Course Schedule</caption>
      <thead>
        <tr>
          <th scope="col">Course</th>
          <th scope="col">Date</th>
          <th scope="col">Level</th>
        </tr>
      </thead>
      <tbody>
        <tr>
          <th scope="row">HTML Fundamentals</th>
          <td>June 10</td>
          <td>Beginner</td>
        </tr>
      </tbody>
    </table>
  </section>

  <section id="register">
    <h2>Register for a Course</h2>
    <form action="/register">
      <label for="full-name">Full Name:</label>
      <input type="text" id="full-name" name="full-name" required>

      <label for="email">Email Address:</label>
      <input
        type="email"

```

```

        id="email"
        name="email"
        required
        aria-describedby="email-help"
    >
    <p id="email-help">We will send your course confirmation to this
email.</p>

    <button type="submit">Register</button>
</form>
</section>
</main>

<footer>
    <p>&copy; 2026 HTML Learning Hub</p>
</footer>
</body>
</html>

```

This is a basic structure.

Your final capstone can add image, media, transcript, aside content, and one justified ARIA example.

25. Capstone ARIA Example: Icon-Only Button

Your capstone should include one carefully justified ARIA example.

Good example:

```

<button type="button" aria-label="Open course search">
    <span aria-hidden="true"><img alt="magnifying glass icon" data-bbox="380 665 400 680"/></span>
</button>

```

Why this is acceptable:

- The button has no visible text
- The icon alone may not be understood by screen readers
- `aria-label` gives the button a clear accessible name
- `aria-hidden` prevents the decorative icon from being announced unnecessarily

Simple rule:

Every ARIA attribute in your capstone must have a clear reason.

26. Capstone Self-Check

Before you finish your capstone, check these points.

Document Structure Check

Ask:

- Does the file start with `<!DOCTYPE html>`?
- Does the `html` element have a `lang` attribute?
- Does the page have a `head` and `body`?
- Is visible content inside the `body`?
- Is metadata inside the `head`?

Semantic HTML Check

Ask:

- Did I use `header` for introductory page content?
- Did I use `nav` for navigation links?
- Did I use `main` for the main content?
- Did I use `section` for meaningful content groups?
- Did I use `article` for self-contained content?
- Did I use `aside` only for related extra content?
- Did I use `footer` for footer information?
- Did I avoid unnecessary `div` elements?

Heading Check

Ask:

- Does the page have one clear `h1`?
- Are main sections using `h2`?
- Are subsections using `h3`?
- Did I avoid skipping heading levels?
- Did I use headings for structure, not visual size?

Form Check

Ask:

- Does every input have a real label?
- Does every `label` for match the correct input `id`?
- Does every input have a useful `name`?

- Did I avoid using placeholder as a label?
- Did I use `required` where needed?
- Did I use `type="email"` for email fields?
- Did I use `aria-describedby` only for helpful extra text?
- Does the `aria-describedby` value match a real `id`?

Table Check

Ask:

- Is the table used for real tabular data?
- Does the table have a `caption`?
- Does the table use `th` for header cells?
- Do column headers use `scope="col"`?
- Do row headers use `scope="row"` where useful?
- Is the table simple and not overly complex?

Image Check

Ask:

- Does each meaningful image have useful `alt` text?
- Did I avoid weak alt text like `alt="image"`?
- Do decorative images use empty `alt=""`?
- If an image is inside a link, does the alt text describe the link action?

Media Check

Ask:

- Does video or audio have controls?
- Does video with speech have captions or subtitles?
- Did I use a `track` element where needed?
- Did I provide a transcript or text alternative?

ARIA Check

Ask:

- Did I use native HTML first?
- Did I avoid unnecessary ARIA?
- Did I avoid using ARIA to replace real HTML?
- Did I remember that ARIA does not add behavior?
- Did I use `aria-label` only when visible text is missing?
- Did I use `aria-labelledby` only when pointing to visible text?
- Did I use `aria-describedby` only for extra descriptions?

- Did I use `aria-hidden` only for decorative content?
 - Did I avoid `aria-hidden` on focusable elements?
 - Did I avoid positive `tabindex` values?
 - Did I avoid `accesskey` unless clearly justified?
-

27. Final Sprint 12 Memory Rules

Memorize these rules:

1. Native HTML first.
 2. ARIA second.
 3. No ARIA is better than bad ARIA.
 4. ARIA gives meaning, not behavior.
 5. Use real `button` for actions.
 6. Use real `a` for links.
 7. Use real `label` for form fields.
 8. Use semantic landmarks like `nav`, `main`, and `footer` before ARIA landmark roles.
 9. Use `aria-label` when there is no visible label.
 10. Use `aria-labelledby` when visible text should label something.
 11. Use `aria-describedby` for help text, hints, warnings, or errors.
 12. Use `aria-hidden="true"` only for content assistive technology should ignore.
 13. Never use `aria-hidden` on focusable content.
 14. Avoid positive `tabindex`.
 15. Be careful with `accesskey`.
 16. Every ARIA attribute must have a reason.
 17. Test accessibility with real keyboard navigation.
 18. Validate your HTML.
 19. Keep the page simple and semantic.
 20. Your final capstone should prove that you can build accessible HTML without depending on JavaScript or React.
-

28. Revision Questions

Answer these without looking at the notes.

1. What does WAI-ARIA mean?
2. Why does ARIA exist?
3. What is the difference between WCAG and WAI-ARIA?
4. What is the most important ARIA rule?
5. Why is a real `button` better than `<div role="button">`?
6. Does ARIA add behavior?
7. What is an ARIA role?

8. What is an ARIA state?
 9. What is an ARIA property?
 10. What does `aria-label` do?
 11. When should you use `aria-label` ?
 12. What does `aria-labelledby` do?
 13. What must `aria-labelledby` point to?
 14. Why should you not use `aria-label` and `aria-labelledby` together?
 15. What does `aria-describedby` do?
 16. What kind of text is good for `aria-describedby` ?
 17. What does `aria-hidden="true"` do?
 18. Does `aria-hidden` hide content visually?
 19. Why is `aria-hidden` dangerous on focusable elements?
 20. Why should positive `tabindex` values be avoided?
 21. Why can `accesskey` cause problems?
 22. What should your final capstone include?
 23. Why should every ARIA attribute have a reason?
 24. What does “native HTML first” mean?
 25. What does “No ARIA is better than bad ARIA” mean?
-

29. Mandatory Practice Coding Block

Warm-Up Drills

Write these small patterns from memory.

1. Real Button

Write a real button for opening a menu.

Expected pattern:

```
<button type="button">Open Menu</button>
```

2. Icon-Only Button with `aria-label`

Write an icon-only search button.

Expected pattern:

```
<button type="button" aria-label="Search">
  <span aria-hidden="true">🔍</span>
</button>
```

3. Section Labelled by Heading

Write a section labelled by a visible heading.

Expected pattern:

```
<h2 id="course-heading">Featured Courses</h2>
<section aria-labelledby="course-heading">
  <p>Choose a course to begin learning HTML.</p>
</section>
```

4. Input with Help Text

Write an email input with help text using `aria-describedby`.

Expected pattern:

```
<label for="email">Email Address:</label>
<input
  type="email"
  id="email"
  name="email"
  aria-describedby="email-help"
>
<p id="email-help">We will send your confirmation to this email.</p>
```

5. Decorative Icon

Write a settings button where the icon is decorative and the visible text explains the button.

Expected pattern:

```
<button type="button">
  <span aria-hidden="true">⚙️</span>
  <span>Settings</span>
</button>
```

30. Guided Capstone Task

Create a file named:

```
accessible-html-capstone.html
```

Build an accessible course catalog page.

Your page should include these parts:

Step 1: Boilerplate

Add:

- `<!DOCTYPE html>`
- `<html lang="en">`
- `head`
- `meta charset="UTF-8"`
- viewport meta
- title
- meta description
- body

Step 2: Header and Navigation

Add:

- `header`
- one clear `h1`
- `nav`
- links to page sections

Step 3: Main Content

Add:

- `main`
- at least three `section` elements
- correct heading hierarchy

Step 4: Course Articles

Add:

- course cards using `article`
- each course should have a heading and paragraph

Step 5: Accessible Image

Add:

- one meaningful image with useful `alt`
- optional decorative image with `alt=""`

Step 6: Accessible Table

Add:

- `table`
- `caption`
- `thead`
- `tbody`
- `th`
- `td`
- `scope="col"`
- `scope="row"`

Step 7: Accessible Form

Add:

- `form action=""` or a practice action value
- labels
- inputs
- `id`
- `name`
- `required`
- `type="email"`
- `aria-describedby` for help text
- submit button

Step 8: Media Alternative

Add:

- video or audio with `controls`
- `track` for captions/subtitles if video has speech
- a short transcript below the media

Step 9: Minimal ARIA

Add only justified ARIA, such as:

- `aria-label` for an icon-only button
- `aria-describedby` for form help text
- `aria-hidden` for decorative icon

Do not add ARIA everywhere.

Step 10: Final Check

Check:

- Valid HTML
- Semantic structure
- Correct headings
- Accessible form
- Accessible table
- Useful alt text
- Media alternatives
- Minimal ARIA
- No fake buttons
- No positive `tabindex`
- No unnecessary `accesskey`
- No React
- No DOM manipulation

31. Final Summary

Sprint 12 completes your HTML foundation.

The main lesson is not “use a lot of ARIA.”

The main lesson is:

Write strong native HTML first. Add ARIA only when it solves a real accessibility problem.

Good HTML already gives structure, meaning, and behavior.

ARIA should only support that structure when something is missing.

Final beginner mindset:

If HTML can do it, use HTML. If HTML cannot fully explain it, then carefully add ARIA.

After Sprint 12, you should be able to build a complete accessible HTML page using semantic structure, forms, tables, media, accessibility patterns, and small careful ARIA additions.